🔥 PART 1 — Topics you MUST master before Monday (only the essentials)

These match EXACTLY what appears in your job description.

1. Authentication + Authorization

Know these 5 things cold:

- JWT (HS256 vs RS256)
- OAuth2 (Access Token)
- OIDC (ID Token + discovery + JWKS) : **OIDC adds an identity layer to OAuth2.**
  You authenticate the user and receive an **ID Token**, which is a JWT with identity claims.
  The provider exposes a **discovery endpoint** that tells clients where all OAuth2/OIDC endpoints are and which signing algorithms they use.
  To verify the ID Token's signature, clients download the provider's **JWKS**, which contains the public keys used to sign tokens.
- RBAC vs ABAC:

Feature	RBAC	ABAC
Access basis	Roles	Attributes (user, resource, environment)
Flexibility	Low	Very high
Policy format	Simple	Complex
Scalability	Limited	Very scalable
Common usage	Admin dashboards, apps with fixed permissions	Enterprises, Zero Trust, regulated systems
Performance	Fast	Slightly slower (policy evaluation)

"Which one is more flexible?"

ABAC is far more flexible because it evaluates policies instead of static roles.

- Microservice-to-microservice auth:

- Service A Service B
- | |
- |---[mTLS handshake]---->|
- |---[JWT token]--------->| verify (sig, exp, scope)
- | |
- |--- request accepted --->|

  Microservice-to-microservice auth ensures that every service call is authenticated and authorized.

  Typically this is done using mTLS(Mutual Transport Layer Security) for service identity at the transport layer and JWT/OAuth2 tokens for application-layer authorization.

  Service A presents either its certificate or a signed token, and Service B verifies the signature, expiry, and scopes before accepting the request.

  This enforces zero-trust between internal services."

  Zero-Trust: ("Never trust. Always verify.")

Expected questions

- *"Difference between RS256 and HS256?"*
- *"How will you validate a JWT in Nginx?"*
- *"JWT vs OAuth2 vs OIDC?"*
- *"How do you invalidate a JWT?"*

---

2. OWASP(Open Web Application Security Project) Top 10 (Secure Coding)

They will definitely ask this because the JD says *"Strong emphasis on software security."*

Memorize:

- SQL Injection:

  SQL Injection is a vulnerability where an attacker inserts malicious SQL into an application's query, allowing them to read, modify, or delete data the application shouldn't allow.

 Bypass authentication

 Read sensitive data

 Modify or delete records

 Drop tables

 Execute administrative commands

 Gain complete control of the database

 Lead to full system compromise

Prevent: It’s prevented using prepared statements, parameterized queries, ORMs, input validation, and least-privilege DB access.
For NoSQL and Neo4j, similar injection risks exist, so we validate input types and use parameterized queries.

- Broken Access Control:

  Broken Access Control happens when an application fails to enforce who can access what, allowing users to act outside their permissions.
  It leads to unauthorized data access, privilege escalation, and account takeover.

  1. IDOR (Insecure Direct Object Reference)

  User can change an ID in the URL:

  GET /api/user/124 ← attacker accesses someone else’s data

  Why?
  Because API trusts the ID and doesn’t check ownership.

---

  2. Role Bypass

  Endpoint contains only client-side role checks:

  if (user.role === 'admin') showAdminButton();

  Attacker calls the admin API directly:

POST /admin/delete-user

Server has no role validation → **broken access control**.

---

3. Missing server-side authorization

Frontend hides “delete” button for non-admins,
but server does **not** enforce it.

---

4. Privilege Escalation

User changes role in their JWT (if unsigned) or in a weak system.

🟩 **What Broken Access Control allows attackers to do**

- Access other users’ data
- Modify or delete resources they shouldn’t
- Become admin (Privilege escalation)
- Access internal APIs
- Abuse hidden endpoints
- Execute operations from other tenants

- XSS :
- **XSS (Cross-Site Scripting) is a vulnerability where an attacker injects malicious JavaScript into a website, which then runs in the victim’s browser.**
  It allows attackers to steal cookies, tokens, sessions, or perform actions as the victim.
- XSS is basically **attacker-controlled JavaScript executing in someone else’s browser**.

Impact of XSS (Say 3–4 in interview)

XSS can:

- Steal session cookies
- Steal JWT tokens

- Modify UI (phishing)
- Perform actions as the user
- Install malware
- Capture keystrokes
- Redirect or deface pages

- CSRF:

  CSRF (Cross-Site Request Forgery) is an attack where a malicious website tricks a logged-in user's browser into sending unwanted requests to a trusted website using the victim's own session, cookies, or authentication tokens.

  In simple words:

  Attacker forces the victim's browser to perform actions the victim did not intend.

  Impact of CSRF

  Unauthorized fund transfers

  Change of password

  Change of email

  Delete account

  Perform admin actions

  Any POST/GET/PUT request the user is allowed to perform

  It's prevented using SameSite cookies, CSRF tokens, double-submit cookies, and by using Authorization headers instead of cookies for API calls."

- Sensitive Data Exposure

  Sensitive Data Exposure occurs when an application accidentally exposes or fails to protect sensitive information such as passwords, personal data, tokens, or financial details.
  It usually results from missing encryption, weak cryptography, insecure storage, or improper data handling.

It’s about **data leakage**, intentional or accidental.

What Data is Considered Sensitive?

Mention these in interview:

Passwords

Tokens (JWT, refresh tokens, API keys)

Personal data (PII: email, phone, address)

Financial data (credit card numbers)

Health data (PHI)

Credentials

Internal configurations

Encryption keys

“Sensitive Data Exposure happens when an application fails to properly protect sensitive information like passwords, tokens, financial data, or personal details.
It’s caused by missing TLS, weak encryption, logging sensitive data, insecure storage, or leaking tokens.
It’s prevented through strong TLS, encryption at rest, secure password hashing (bcrypt/argon2), proper secret management, token hygiene, and avoiding logging or exposing sensitive info in URLs.

- SSRF

SSRF (Server-Side Request Forgery) is a vulnerability where the attacker tricks the server into making HTTP requests to internal or unintended systems.
The attacker uses your server as a proxy to reach internal networks, cloud metadata, or restricted services.

In simple words:

Attacker forces *your server* to make requests *on their behalf*.

SSRF happens when an attacker tricks the server into making HTTP requests to internal systems or cloud metadata endpoints.
It's extremely dangerous because the server often has access the attacker doesn't.
Prevention includes strict allowlists, blocking internal IP ranges, validating URLs, disabling redirects, enforcing network-level restrictions, and never fetching user-supplied URLs without validation.

- Insecure DeserializationL:

Insecure Deserialization occurs when untrusted or user-controlled data is deserialized into objects.
Attackers exploit this to execute malicious code, modify application logic, or escalate privileges.
Simply put:
If you deserialize untrusted input → attacker can make your app run their code.

🟦 How Deserialization Works (quick recap)

Serialization = convert object → bytes/string
Deserialization = bytes/string → object
If user controls that data → they control the object.

Insecure Deserialization happens when untrusted input is deserialized into objects.
This is dangerous because it can lead to remote code execution, privilege escalation, or tampering with application logic.
It's prevented by never deserializing untrusted data, using safe formats like JSON instead of binary serialization, applying strict allowlists, signing serialized objects, and disabling risky deserialization features.

- XXE:

XXE (XML External Entity) is a vulnerability where an XML parser processes user-controlled XML that defines external entities.
This allows attackers to read server files, perform SSRF, or cause denial of service.**

In short:

Attacker puts a malicious entity inside XML → parser fetches or expands it → data leaks or server gets attacked.

XXE happens when an XML parser processes malicious external entities.

Attackers can read server files, perform SSRF, or cause DoS using entity expansion.

Prevention involves disabling external entities, disabling DTDs, using secure XML parser configurations, validating input, or avoiding XML for untrusted data.

- Security Misconfiguration;

  Security Misconfiguration occurs when systems, servers, or applications are configured with insecure settings, default configurations, unnecessary features, or exposed services, leading to vulnerabilities.

  In simple words:

  You left something open that should be locked.

  Security Misconfiguration happens when systems run with insecure settings — like default credentials, exposed admin interfaces, unnecessary open ports, verbose errors, missing security headers, or misconfigured cloud buckets.
  It is prevented by disabling unused features, applying least privilege, keeping software updated, securing configurations, using proper headers, rotating secrets, and regularly reviewing and hardening server/NGINX/Kubernetes settings.

Expected questions:

- *"What is SQL injection? How do you prevent it?"*
- *"XSS prevention?"*
- *"What is CSRF and how to stop it?"*

---

3. NGINX Security

Based on your conversations, you already know ~50%.

Add these:

- access_by_lua_block:

access_by_lua_block is an OpenResty/NGINX directive that allows you to run Lua code during the *access phase* of the NGINX request lifecycle, mainly for authentication, authorization, and security checks before the request is passed to the upstream service.

In simple words:

It lets you run security logic (JWT validation, role checks, header checks) inside NGINX *before* proxying the request.

🟦 **Why it exists**

Because NGINX doesn't natively understand:

JWT tokens

OAuth2

OpenID Connect

role-based access

custom authorization logic

access_by_lua_block gives NGINX the flexibility to implement **custom security** in Lua.

Q: Why use access_by_lua_block instead of other directives?

Answer:

Because the access phase is where NGINX enforces allow/deny decisions. access_by_lua runs before proxying, so it is perfect for JWT validation, OAuth2 introspection, and RBAC enforcement.

Q: What happens if the Lua code returns 401 or 403?

Answer:

NGINX immediately returns the response and never forwards the request to the upstream service.

Q: What if Lua code throws an error?

Answer:

NGINX will treat it as an internal failure and return 500 unless you handle errors in Lua.

Q: Can access_by_lua_block modify request body?

Answer:

Yes, but it's not recommended. The goal is access control, not request transformation.

🟩 **20-Second Interview Answer**

"access_by_lua_block runs Lua code during NGINX's access phase to enforce authorization and authentication. It's used for validating JWTs, checking roles, calling OAuth2 introspection, and implementing custom security logic before the request reaches upstream services. If the Lua script denies the request, NGINX returns 401/403 immediately."

- auth_request:

  auth_request is an NGINX directive that forwards a sub-request to an external authentication service.
  Based on the sub-request's HTTP status code (200/401/403), NGINX allows or denies the original request.

  In simple words:

  NGINX asks another service → "Is this request allowed?" → and enforces the answer.

🟦 **Why auth_request exists**

Because many companies want **centralized authentication** outside NGINX itself.

Instead of writing Lua code inside NGINX, you can:

send a request to an **auth microservice**

auth service validates JWT / OIDC / session

responds with **200 OK** (allow) or **401/403** (deny)

NGINX then:

continues if allowed

blocks if denied

No authorization logic inside NGINX → cleaner architecture.

auth_request lets NGINX delegate authentication to a separate microservice.
NGINX sends a sub-request to an /auth endpoint, and if it returns 200, the request proceeds; if 401 or 403, NGINX blocks it.
This allows centralized JWT/OIDC validation and keeps NGINX configuration clean while maintaining Zero Trust between gateway and services

- CORS
- Rate limiting:

  Rate limiting restricts how many requests a client can make in a given time window.
  It protects APIs and backend systems from abuse, brute-force attacks, and traffic spikes.

  It is a core part of **API security**, **DoS protection**, and **Zero Trust**.

🟦 **Why Rate Limiting is needed**

Rate limiting protects against:

Brute-force login attempts

API abuse

DoS attacks

Bots / scrapers

Misbehaving clients

Cost spikes (cloud billing)

Flooding microservices

Without rate limiting, one client can overload your entire system.

Rate limiting restricts how many requests a client can make within a specific time window.
It protects APIs from brute-force attacks, DoS attempts, and abuse.
In NGINX, we use limit_req_zone and limit_req to rate-limit per IP, token, or user, returning 429 when limits are exceeded.
More advanced distributed systems use Redis or API gateways for global rate limiting.

Rate Limiting per… (Common strategies)

✓ per IP address

Useful for public APIs.

✓ per user / per token

Identity-based rate limiting.

✓ per API key

SaaS companies rely heavily on this.

✓ per endpoint

e.g., /login usually has stricter limits.

✓ per tenant

Multi-tenant systems — extremely useful.

- TLS termination basics:

- **TLS termination is the process where an incoming HTTPS connection is decrypted at a gateway or load balancer (like NGINX), and the internal traffic is forwarded as plain HTTP or re-encrypted.**
- In simple words:
- **The edge server handles HTTPS → decrypts → passes decrypted traffic to backend services.**
- ---
- 🟦 **Why TLS Termination is needed**
- TLS termination allows:
- Offloading expensive TLS handshake work from backend services
- Centralized certificate management
- Enforcing consistent TLS policies
- Easier scaling + performance
- Secure entry point for microservices
- ---
- 🟩 **Where TLS Termination happens**
- At:
- API Gateway (NGINX, Kong, Envoy)
- Load balancer (AWS ALB/NLB, GCP LB)
- Kubernetes Ingress Controller
- Reverse proxy
- Backend services often run on **HTTP**, not HTTPS.
- ---
- 🟧 **Flow: How TLS Termination Works**
- 1️⃣ **Client sends HTTPS request:**
- https://api.example.com

- 2️⃣ **NGINX performs TLS handshake:**
- Presents certificate
- Uses private key to decrypt session key
- Establishes secure session
- 3️⃣ **After handshake, encrypted traffic becomes:**
- HTTP request inside NGINX
- 4️⃣ **NGINX forwards decrypted traffic to internal upstream:**
- proxy_pass http://backend-service;
- Backend receives *plain HTTP* or optionally re-encrypted HTTPS.
- ---
- 🟪 **TLS Termination vs TLS Passthrough vs TLS Re-encryption**

Mode	Meaning	Used for
TLS Termination	NGINX decrypts and proxies HTTP	Most APIs
TLS Passthrough	NGINX sends encrypted traffic directly to backend	mTLS, certificate-bound apps
TLS Re-encryption	NGINX decrypts AND re-encrypts to backend	Highly secure internal networks

- ---
- 🟥 **TLS Handshake (Simple Explanation)**
- Steps:
- Client → Server: “Hello, give me your certificate.”
- Server → Client: Certificate (contains public key)
- Client verifies certificate trust
- Client generates session key
- Client encrypts session key with server’s public key

Server decrypts using private key

Both have shared symmetric key

All communication now encrypted via symmetric encryption (AES)

This is why:

TLS is fast (symmetric crypto)

Secure (asymmetric only for handshake)

🟫 **Important TLS Concepts (Interview Must-Know)**

✓ Certificates

Contain:

Public key

Expiry

Issuer

Domains covered (SAN)

✓ Private Key

Only on server.
Never shared.

✓ Cipher Suites

Algorithms used for:

Key exchange

Encryption

MAC

✓ TLS Versions

Use **TLS 1.2+ or TLS 1.3**
Disable:

TLS 1.0

- TLS 1.1
- weak ciphers
- ---
- 🟦 **NGINX TLS Termination Example**
- server {
- listen 443 ssl;
- server_name api.example.com;
-
- ssl_certificate /etc/nginx/certs/api.crt;
- ssl_certificate_key /etc/nginx/certs/api.key;
- ssl_protocols TLSv1.2 TLSv1.3;
-
- location / {
- proxy_pass http://backend-service;
- }
- }
- This means:
- Client communicates over HTTPS
- Backend communicates over HTTP
- ---
- 🟩 **Why Backend Often Uses HTTP After TLS Termination**
- Because:
- Internal network is protected (VPC, cluster mesh, firewall)
- Reduces CPU load
- Easier debugging
- Simpler service configuration

For sensitive environments, you can do:
👉 **TLS Re-encryption** to backend (HTTPS to HTTPS)

🟨 **TLS Termination in Kubernetes**

Ingress Controller (NGINX, Traefik, Envoy) terminates TLS.
Services inside cluster run over HTTP.

🟥 **TLS Termination + mTLS**

At gateway:

user-facing TLS (normal HTTPS)
Inside cluster:

mTLS between microservices (Istio)

This is **Zero Trust** architecture.

🟩 **20-Second Interview Answer**

“TLS termination means NGINX or a gateway decrypts incoming HTTPS traffic, performs the TLS handshake, and forwards the internal traffic to upstream services, usually over HTTP.
It centralizes certificate management, offloads crypto work, and simplifies microservice design.
It is essential for secure API gateways and can be combined with TLS passthrough or mTLS depending on architecture.”

- Reverse proxy security:

  A reverse proxy is a server placed in front of backend services that receives client requests, applies security controls, and forwards them to internal services.

  NGINX, Envoy, HAProxy, and Traefik are the most common examples.

🔥 Why Reverse Proxy Security Matters

Because the reverse proxy is the **first line of defense** for APIs and microservices.

If the proxy is misconfigured:

- Attackers hit internal services directly
- Header spoofing can bypass auth
- Rate limiting fails
- TLS weakens
- Sensitive upstream systems become exposed

Reverse proxy security is essential.

🟦 Key Security Responsibilities of a Reverse Proxy

These are the points interviewers expect from a senior engineer.

✓ 1. TLS termination (secure)

- Only allow TLS 1.2/1.3
- Disable weak ciphers
- Enforce HSTS
- Use strong certificates

✓ 2. Authentication & Authorization

Implement using:

- JWT validation (Lua / auth_request)
- OAuth2 / OIDC
- mTLS
- API keys
- Role enforcement

NGINX must:

- block unauthorized requests
- forward only safe traffic

✓ 3. Rate limiting & DoS protection

Prevent brute-force and traffic spikes.

Use:

- limit_req (rate limiting)
- limit_conn (limit connections)
- http2_max_concurrent_streams

✓ 4. Header Sanitization

Remove dangerous or spoofed headers:

- X-Forwarded-For
- Host
- X-Real-IP
- custom auth headers

NGINX should set correct headers itself, not trust client-supplied ones.

Example:

proxy_set_header Host $host;

proxy_set_header X-Real-IP $remote_addr;

proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

✓ 5. Request Filtering & Validation

Block:

- oversized requests (client_max_body_size)
- suspicious methods (TRACE, CONNECT)

- invalid paths
- malformed headers
- SQLi payload patterns (through WAF)

✓ 6. CORS Enforcement

Allow only legitimate origins.

Access-Control-Allow-Origin: https://trusted.com

Never set:

Access-Control-Allow-Origin: *

Unless truly public.

✓ 7. Hide Backend Details

Do NOT expose:

- upstream IPs
- NGINX version
- server signature

Use:

server_tokens off;

✓ 8. Path Normalization

Prevent bypasses like:

- path traversal
- double encoding
- URL rewriting attacks

✓ 9. Firewall Integration

Reverse proxy should sit behind:

- network firewall
- WAF
- DDoS protection

Cloud providers often combine these.

✓ 10. Logging, Monitoring, and Alerts

Log:

- failed logins
- rate limit hits
- suspicious paths
- JWT validation failures

Send logs to:

- ELK
- Splunk
- Datadog

For alerting and threat detection.

Reverse Proxy Attack Scenarios (Must mention in interview)

1. Header Spoofing

Attacker sends:

X-Forwarded-For: 127.0.0.1

Backend thinks request is internal → access bypass.

Fix:

Do not trust client headers.

---

2. Host Header Injection

Attacker sets:

Host: evil.com

Can break routing, caching, or cause SSRF.

Fix:

Validate Host header.

---

3. Misconfigured proxy_pass leaking file system

Typical mistake:

```
location / {
   proxy_pass http://localhost;
}
```

Trailing slashes matter! Can distort path rewriting.

---

4. Open Redirect via reverse proxy

Unvalidated redirects end up exposing user to phishing.

---

5. Exposed admin endpoints

If /admin accidentally reachable through proxy → total compromise.

🟧 **Best Practices for Reverse Proxy Security**

🔐 **1. Use strong TLS**

TLS1.2 or TLS1.3 only.

🔎 **2. Validate and sanitize headers**

Prevent spoofing.

🎫 **3. Enforce proper authentication**

JWT/OIDC checks in access_by_lua or auth_request.

🚦 **4. Rate limit & block bots**

Use limit_req, limit_conn.

🛡️ **5. Enable WAF rules**

ModSecurity / OpenResty WAF.

🖌️ **6. Strip sensitive headers**

Never pass:

- Authorization header from client unchanged (if not intended)
- cookies that backend shouldn't see

🔒 **7. Deny access to internal paths**

Block /internal/* using:

```
location /internal {
  deny all;
}
```

📏 **8. Set request size & timeout limits**

To mitigate DoS.

🟫 **NGINX Example — Secure Reverse Proxy**

```
server {
  listen 443 ssl;
  server_name api.example.com;


  server_tokens off;


  ssl_protocols TLSv1.2 TLSv1.3;
```

```
    limit_req_zone $binary_remote_addr zone=api_limiter:10m rate=10r/s;

    location / {
        limit_req zone=api_limiter nodelay;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        proxy_pass http://backend-service;
    }
}
```

🟦 20-Second Interview Answer

"A reverse proxy sits in front of backend services and acts as the first security gateway.
It handles TLS termination, authentication, rate limiting, CORS, header sanitization, request validation, and hides backend details.
In NGINX, this is done using directives like auth_request, access_by_lua_block, limit_req, CORS rules, secure headers, and proxy security configurations.
Securing the reverse proxy is critical because if it's misconfigured, the entire backend becomes exposed."

- Header sanitization:
- **Header sanitization is the process of validating, filtering, and rewriting HTTP headers at a reverse proxy or gateway to prevent spoofing, privilege escalation, and security bypasses.**

- In simple words:
- **Never trust headers coming from the client — sanitize them before forwarding to backend services.**

🟦 Why Header Sanitization is Critical

Attackers can manipulate headers to:

- spoof IP addresses
- bypass authentication
- elevate privileges
- trick backend services into thinking request is internal
- break routing logic
- inject harmful data

Since headers carry sensitive information (Authorization, Set-Cookie, X-Forwarded-For), sanitizing them is required for secure microservices.

🟩 Common Attacks Prevented by Header Sanitization

1️⃣ IP Spoofing via X-Forwarded-For

Attacker sends:

X-Forwarded-For: 127.0.0.1

Backend sees request as internal → bypasses firewall rules.

2️⃣ Host Header Injection

Attacker sets:

Host: evil.com

Used for:

- cache poisoning
- password reset poisoning
- routing bypass

3️⃣ Header-based Authentication Bypass

Example:

X-User: admin

Some legacy services trust this.

4️⃣ CORS Header Injection

Attacker sets:

Origin: https://evil.com

If server echoes it → cross-domain data theft.

5️⃣ CRLF Header Injection

Attackers inject:

%0d%0aSet-Cookie: admin=true

6️⃣ Overwriting proxy headers

Client sends:

X-Real-IP: 10.0.0.1

Should be overwritten by NGINX.

🟧 What Headers Must be Sanitized (Important for interview)

✓ Never trust from client:

- X-Forwarded-For
- X-Real-IP
- X-Forwarded-Proto

- X-Forwarded-Host
- X-User / X-Role (custom)
- Authorization (validate, don't blindly forward)
- Host

  These should be **overwritten** by NGINX.

🟦 How NGINX Does Header Sanitization

✓ 1. Reset IP headers

```
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

This overwrites any attacker-supplied values.

✓ 2. Validate Host header

```
if ($host !~* ^(api\.example\.com)$) {
   return 400;
}
```

✓ 3. Remove dangerous custom headers

```
proxy_set_header X-User "";
proxy_set_header X-Role "";
```

Or:

```
proxy_hide_header X-User;
```

✓ 4. Block unknown or malformed headers

```
if ($http_x_suspect_header) {
```

return 403;

}

✓ 5. Do not forward client Authorization header unless required

Example:
Gateway should validate JWT, then pass only derived identity.

✓ 6. Enforce only safe headers passed upstream

Explicit allowlist:

proxy_pass_header Content-Type;

proxy_pass_header Authorization; # if needed

🟧 Advanced — Header Normalization

Normalize:

- case
- whitespace
- duplicated headers
- line folding
- special characters

Example:

X-FoRwarded-For → x-forwarded-for

Prevents bypass during routing or in auth filters.

🟫 Context: Microservices & Zero Trust

In a zero-trust architecture:

- authentication happens at gateway

- identity must not be spoofed
- IP headers must be accurate
- only validated identity claims forwarded

  Header sanitization enforces boundary trust.

🟧 **Example — Secure NGINX Header Sanitization**

proxy_set_header Host $host;

proxy_set_header X-Real-IP $remote_addr;

proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

proxy_set_header X-Forwarded-Proto $scheme;

proxy_set_header X-User "";

proxy_set_header X-Role "";

proxy_hide_header X-Powered-By;

🟩 **What to say in the interview (20-second answer)**

“Header sanitization ensures that untrusted or spoofed headers coming from clients are removed, validated, or overwritten before reaching backend services. It prevents IP spoofing, host header injection, privilege escalation, and bypassing authentication.
In NGINX, we sanitize headers using proxy_set_header, block malicious headers, normalize values, and enforce strict allowlists.
This is essential in Zero Trust and microservice security.”

- Upstreams + keepalive:

  🔵 **What are NGINX Upstreams? (Perfect Interview Definition)**

An NGINX upstream is a group of backend servers that NGINX proxies requests to.
It enables load balancing, failover, service discovery, and connection reuse.

Example:

```
upstream user-recordings-sync {
  server 127.0.0.1:7725;
  keepalive 50;
}
```

This defines a backend *pool* for NGINX to proxy requests to.

🟦 Why Upstreams Exist

Upstreams allow NGINX to:

- load balance between multiple backend instances
- retry on failure
- health check backends
- manage keepalive connections
- unify connection handling for microservices

They turn NGINX into a **reverse proxy + load balancer**.

🟩 How Upstreams Work (Simple Flow)

1. Client hits:
2. https://api.example.com
3. NGINX receives request
4. Applies security rules (JWT auth, rate limiting, etc.)
5. NGINX proxies request to **upstream backend**:
6. http://user-recordings-sync

7. Backend responds → NGINX returns result to client

   Upstream group decides **which backend** to pick.

🟧 **Load Balancing Methods (Must-Know)**

Common upstream load balancing modes:

- **Round Robin** (default)
- **Least connections**
- **IP hash**
- **Least time**

Example:

```
upstream api {
  least_conn;
  server backend1:8080;
  server backend2:8080;
}
```

🟪 **What is keepalive? (Perfect Interview Definition)**

keepalive defines how many idle, persistent connections NGINX should keep open to upstream servers.
It improves performance by reusing TCP connections instead of opening a new connection per request.

Example:

```
keepalive 50;
```

Meaning:

- Keep up to 50 connections **open and reused**
- Avoid expensive TCP + TLS handshakes
- Reduce latency

- Reduce CPU cost on backend

🟥 **Why keepalive is important (Interview Points)**

✓ 1. Performance

Avoids overhead of creating new connections.

✓ 2. Lower latency

Reusing warm connections is faster.

✓ 3. Backend scalability

Backend servers aren't flooded with constantly new connections.

✓ 4. Better throughput

High TPS microservices rely heavily on keepalive.

✓ 5. Essential for high-traffic APIs

Without it, NGINX becomes bottleneck.

🟦 **Common Mistake (Interview Expected)**

Misconfigured keepalive = connection explosion + backend overload

If you have:

keepalive 0

Then NGINX:

- Opens new TCP connection for every request
- Closes them immediately
- Causes huge overhead
- Limits throughput severely

🟩 **Security Perspective on Upstreams + keepalive**

🔐 **1. Limits exposure**

Backend services are NOT exposed to the internet.
Only NGINX talks to them.

🔐 **2. Controls origin IP**

Backend sees NGINX IP, not attacker IP →
NGINX sanitizes headers before upstream.

🔐 **3. Reduces attack surface**

Backends never deal with:

- TLS handshake
- public IP traffic
- brute-force attempts
- malformed requests
  All handled by NGINX.

🔐 **4. Connection exhaustion protection**

keepalive prevents resource exhaustion on backend.

🔐 **5. Zero Trust support**

Upstreams allow NGINX to enforce:

- JWT validation
- rate limiting
- CORS
- mTLS at edge

  Before anything reaches microservices.

🟧 **Advanced: Upstream Keepalive Requirements**

To work correctly:

- proxy_http_version 1.1

- Connection header must be cleared

  Example:

  location / {

    proxy_http_version 1.1;

    proxy_set_header Connection "";

    proxy_pass http://user-recordings-sync;

  }

  Otherwise keepalive **won't work**.

🟫 **Full Example (Production-Ready)**

upstream user-recordings-sync {

  server 127.0.0.1:7725;

  keepalive 50;

}

server {

  listen 443 ssl;

  location /cdvr/admin/v1 {

    proxy_http_version 1.1;

    proxy_set_header Connection "";

    proxy_set_header Host $host;

    proxy_set_header X-Real-IP $remote_addr;

    proxy_pass http://user-recordings-sync;

  }

}

🟧 **20-Second Interview Answer**

“NGINX upstreams are backend server pools used for load balancing and high availability.
The keepalive directive keeps persistent connections open so NGINX can reuse them instead of opening new TCP connections for every request, which significantly improves performance and reduces backend load.
Upstreams + keepalive are essential for high-scale microservices and also improve security by isolating backend services behind the proxy.”

- JWT verification flow in NGINX:

  JWT Verification Flow in NGINX (Perfect Summary)

  NGINX does NOT natively understand JWTs.
  You must implement JWT verification using Lua (OpenResty) or an external auth service via auth_request.
  The flow validates signature, expiry, issuer, audience, and role claims before proxying to upstream.

  Now let's break down the full flow.

🟦 **JWT Verification Flow – Step by Step**

1️⃣ **Client sends request with JWT**

Authorization: Bearer <JWT>

2️⃣ **NGINX extracts the header inside access phase**

Using:

- access_by_lua_block
  or
- auth_request /auth

3️⃣ **NGINX/Lua parses the JWT**

JWT = header.payload.signature

Lua code/base64 decode extracts:

- header JSON
- payload JSON
- signature (base64url)

Example payload fields:

```
{
 "sub": "user123",
 "role": "adminRole#All",
 "aud": "cdvr-api",
 "iss": "https://idp.example.com",
 "exp": 1715680000
}
```

4️⃣ NGINX loads JWKS (public keys)

For **RS256** signed tokens:

Lua or auth-service fetches JWKS from:

https://idp.example.com/.well-known/jwks.json

JWKS contains the **public key** needed to verify JWT signature.

This is required for:

- RS256
- RS512
- ES256

5️⃣ Verify the signature

Lua code:

- find matching kid in JWT header
- get corresponding public key from JWKS
- use RSA verify algorithm

  If signature invalid → **401 Unauthorized**.

  NGINX stops the request right here.

6️⃣ Validate standard claims

exp

Reject if expired.

nbf

Reject if token not active yet.

iss

Must match configured issuer.

aud

Must match expected API audience.

If any invalid → 401.

7️⃣ Validate role/permissions

Your config specifically verifies:

verify("adminRole#All")

Meaning:

- only tokens containing "adminRole#All" in roles claim may proceed.

Other examples:

- role=viewer
- scope=read:data
- permissions=["read", "write"]

If missing → **403 Forbidden**.

8️⃣ If JWT valid → NGINX proxies to upstream

After successful verification:

proxy_pass http://user-recordings-sync;

Backend service sees **only validated traffic**.

🟩 Two Ways NGINX Can Verify JWT

1. Lua-Based JWT Verification (access_by_lua_block)

This is used in your config:

```
access_by_lua_block {
  if ngx.req.get_headers()["sni-external-request"] ~= nil then
    require("verify_role_with_openid").verify("adminRole#All")
  end
}
```

✓ Flow:

- Lua reads Authorization header
- Parses JWT
- Retrieves JWKS
- Verifies signature
- Validates exp, iss, aud
- Checks role (adminRole#All)
- Returns 401/403 if invalid
- Otherwise NGINX continues

✓ Pros:

- Very fast

- No external network hop
- Low latency
- No need for extra microservice

✓ **Cons:**

- Logic lives in Lua (less maintainable)
- Harder to centralize
- Requires Lua + OpenResty knowledge

---

2. Auth-Service Based Verification (auth_request)

Alternate method:

```
location /api/ {
  auth_request /auth;
  proxy_pass http://backend;
}

location /auth {
  internal;
  proxy_pass http://auth-service/validate;
}
```

✓ **Flow:**

- NGINX forwards sub-request to auth-service
- auth-service checks JWT (Java/Go/Python service)
- Returns:
    - **200 → allowed**
    - **401 → unauthorized**
    - **403 → forbidden**

✓ **Pros:**

- Centralized logic
- Language flexibility
- Easier maintenance
- Better logging and auditing

✓ **Cons:**

- Slightly slower
- Requires another service

🟦 **How NGINX Rejects Bad Tokens**

Issue	Status Code
Missing Authorization header	401
Invalid signature	401
Expired exp	401
Wrong iss or aud	401
Role missing	403
Token tampered	401
No Bearer token	400

This is industry standard.

🟧 **NGINX JWT Verification in Diagram**

Client → NGINX → [Lua or Auth Service]

|

| verifies JWT

|

200 OK / 403 / 401

|

If OK → proxy_pass → upstream

🟫 **What Must Be Verified in JWT (Interview Must-Have)**

✓ Signature

Verify with RSA public key from JWKS.

✓ Claims

- exp
- iss
- aud

✓ Role/Scope claims

- roles
- permissions
- scope

✓ Token is not tampered

Compare signature.

🟥 **Why JWT Verification Must Happen at NGINX (Important)**

- Protect backend from invalid requests
- Offload heavy cryptographic checks from microservices
- Enforce consistent access control
- Improve performance
- Reduce attack surface
- Works with Zero Trust

🟧 **20-Second Interview Answer**

"NGINX verifies JWTs by extracting the token, decoding header and payload, loading the correct public key from JWKS, verifying the RS256 signature, and validating claims like exp, iss, aud, and roles.
If verification fails, NGINX returns 401 or 403; if valid, it forwards the request to the upstream.
This is done using access_by_lua_block for inline Lua verification or auth_request for centralized auth microservices."

Expected questions:

- *"How do you secure NGINX endpoints?"*
- *"What happens inside access_by_lua?"*
- *"How do JWT keys rotate in NGINX?"*

---

4. API & Microservice Security

Know:

- mTLS:

  mTLS (Mutual TLS) is a security mechanism where **both the client and the server authenticate each other using certificates** during the TLS handshake.
  It ensures secure, authenticated, and encrypted communication between services.

  In simple words:

  TLS = server proves who it is
  mTLS = BOTH sides prove who they are

🟦 **Why mTLS Is Used**

mTLS prevents:

- Unauthorized services calling internal APIs
- Impersonation attacks

- MITM attacks
- Lateral movement inside microservice clusters
- Fake or compromised services making calls

It is a key part of **Zero Trust** architecture.

🟩 **mTLS Handshake — What Actually Happens**

During a normal TLS handshake:

- Server shows **its certificate**
- Client verifies it
- Connection becomes encrypted
- BUT **client identity is not verified**

During **mutual TLS**:

1. Server shows certificate → client verifies
2. Client shows certificate → server verifies
3. Both confirm each other's identity
4. A secure, encrypted session is established

This ensures:

- "I know who the server is."
- "I know who the client is."

🟧 **Where mTLS Is Used (Important in Interview)**

✓ **Service-to-service authentication**

Inside microservices environments:

- Kubernetes
- Service Mesh (Istio, Linkerd, Consul)
- Internal APIs

- Financial systems
- IoT devices
- Backend data pipelines

✓ Protecting internal APIs

Example:
user-service can only talk to order-service if both have valid certificates.

✓ Zero Trust Networks

Identity is always verified even inside private networks.

🟪 **How mTLS Works in Microservices**

1️⃣ **Each service gets:**

- A **private key**
- A **certificate signed by trusted CA**

2️⃣ **Certificates are issued by:**

- Private PKI
- Istio Citadel
- Vault PKI
- AWS ACM Private CA
- Custom CA

3️⃣ **When service A calls service B:**

- Both present certificates
- Both validate each other's cert using shared CA
- Only then traffic flows

🟥 **mTLS in NGINX / API Gateways**

Server requiring client certificates:

ssl_verify_client on;

ssl_client_certificate /etc/nginx/ca.crt;

Meaning:

- If client has no certificate → 400/401
- If cert is invalid → rejected

NGINX as mTLS client:

proxy_ssl_certificate /etc/nginx/client.crt;

proxy_ssl_certificate_key /etc/nginx/client.key;

proxy_ssl_trusted_certificate /etc/nginx/ca.crt;

This allows NGINX to authenticate to backend services.

🟫 **Benefits of mTLS**

✓ **Strong authentication**

Identity verified by certificates, not passwords or tokens.

✓ **Prevents impersonation**

Fake services cannot connect.

✓ **Transport security**

TLS encrypts the connection.

✓ **No shared secrets**

Private keys never travel.

✓ **Automatic rotation (with service mesh)**

Istio rotates certs automatically every 24 hours.

🟧 **Drawbacks (Interviewers love this)**

- Certificate management complexity

- Requires private PKI or mesh
- Harder for external clients
- Increases maintenance overhead

But for internal microservices, it's the **gold standard**.

🟩 **mTLS vs TLS vs JWT (Important Comparison)**

Feature	TLS	mTLS	JWT
Verifies server?	✓	✓	❌
Verifies client?	❌	✓	✓ (via token)
Encryption	✓	✓	❌
Stateless	✓	✓	✓
Best for	Browser→Server	Service→Service	API Authorization

mTLS authenticates the caller
JWT authorizes what the caller can do

They often work **together** in Zero Trust.

🟦 **When to Use mTLS**

Use mTLS when:

- Services are internal
- High-security environment
- You want identity at transport layer
- JWT validation alone is not enough
- Preventing impersonation is required

Microservices often use:

- mTLS for **identity**
- JWT for **authorization**

🟧 **Example Architecture with mTLS**

Client ——TLS——> API Gateway ——mTLS——> Microservices

|

JWT auth

🟦 **20-Second Interview Answer**

“mTLS is mutual TLS where both server and client authenticate each other using certificates.
It provides encrypted communication plus strong identity verification at the transport layer.
It’s widely used for microservice-to-microservice authentication in Zero Trust environments.
Gateways or meshes like Istio automatically handle certificate issuance, rotation, and validation.”

- API Gateway concepts:

  What is an API Gateway? (Perfect Interview Definition)

  An API Gateway is the single entry point for all client traffic in a microservices architecture.
  It handles routing, security (auth, authz), traffic control, observability, and request/response transformation before forwarding traffic to backend services.

  In simple terms:

  API Gateway = Reverse Proxy + Security Engine + Traffic Manager + API Management Layer.

🟦 Why API Gateways Exist (Must-Mention Points)

Microservices introduce:

- many internal services
- inconsistent endpoints
- diverse authentication rules
- multiple versions
- scattered security logic

An API Gateway centralizes:

- Authentication
- Authorization
- Rate limiting
- TLS termination
- Logging
- Routing
- Request transformation
- Caching

It protects backend microservices and simplifies clients.

🟩 Core API Gateway Responsibilities

These **8 categories** are exactly what interviewers expect.

1️⃣ Routing & Request Forwarding

- Direct /users → user-service
- Direct /orders → order-service
- Support versioning: /v1, /v2
- Canary and blue/green routing

2️⃣ Authentication (Identity Verification)

Gateways verify identity using:

- JWT (RS256)
- OAuth2
- OIDC
- API keys
- mTLS
- SSO providers

All requests must be authenticated **before** hitting backend.

3️⃣ Authorization (Permissions)

Gateways enforce:

- RBAC (admin, viewer, support)
- ABAC (department, tenant ID)
- JWT scopes & roles
- Policy engines (OPA)

If request unauthorized → 403 at gateway level.

4️⃣ Rate Limiting & Throttling

Prevent:

- brute-force attacks
- bots
- API abuse
- DoS attempts

Forms:

- per-IP
- per-token
- per-API key
- per-tenant

  Gateways return 429 Too Many Requests.

5️⃣ Security Enforcement

- TLS termination
- Header sanitization
- CORS enforcement
- WAF rules
- Input validation
- Bot filtering
- IP allow/block lists
- JWT verification

  Microservices never deal with raw internet traffic.

6️⃣ Traffic Management

- retries
- timeouts
- circuit breakers
- load balancing
- bulkhead isolation
- connection pooling
- request shaping

  This prevents cascading failures.

7️⃣ Request/Response Transformation

Gateways can:

- rewrite URLs
- modify headers
- convert XML ↔ JSON
- mask sensitive fields
- enforce schema validation
- compress responses

8️⃣ Caching

- full response caching
- partial caching
- ETag validation
- microservice response caching

Reduces backend load significantly.

9️⃣ Observability

Gateways generate:

- structured logs
- metrics (TPS, latency, 4xx/5xx counts)
- distributed tracing headers (x-request-id, x-trace-id)

Integrations:

- ELK
- Prometheus/Grafana
- Jaeger

- OpenTelemetry

🟧 **API Gateway Architecture (Interview Diagram)**

Client

|

v

API Gateway (NGINX / Kong / Envoy / APIM)

|

|— Authentication

|— Authorization

|— Rate Limiting

|— Header Sanitization

|— JWT verification

|— Traffic Shaping

|

v

Backend Microservices

Backend services receive **only safe, validated, authenticated traffic**.

🟫 **Real API Gateway Products (Know These)**

Open Source:

- NGINX / OpenResty
- Kong Gateway
- Envoy Proxy
- Traefik
- KrakenD

- Ory Oathkeeper

Cloud:

- AWS API Gateway
- Azure API Management
- GCP API Gateway
- Cloudflare

Service Mesh Gateways:

- Istio Ingress Gateway
- Linkerd
- Consul

🟥 **API Gateway vs Reverse Proxy (Important Question)**

Reverse Proxy	API Gateway
Basic routing	Advanced routing & versioning
No auth	Built-in JWT/AuthZ/OAuth2
No API lifecycle	API key mgmt, quotas
Minimal security	WAF, CORS, mTLS, rate limits
L3/L4	L7 (application layer)
Stateless	Stateful & policy-driven

🟦 **Security Features Expected from an API Gateway**

- JWT signature verification
- OIDC discovery & JWKS
- API keys & HMAC signatures
- RBAC/ABAC enforcement

- Rate limiting/throttling
- mTLS for internal services
- CORS controls
- WAF rules
- Header sanitization
- Input validation (JSON schema)
- Threat detection + bot filtering

  This is what makes it essential in a **Zero Trust** architecture.

🟩 **API Gateway + Service Mesh (Interview Insight)**

Gateway = north-south traffic (client → service)
Mesh = east-west traffic (service → service)

You need **both** for a secure microservice platform.

🟧 **20-Second Interview Answer**

"An API Gateway is the single entry point for microservices.
It handles routing, authentication, authorization, rate limiting, TLS termination, header sanitization, request/response transformations, and logging.
It protects backend services by enforcing consistent security and traffic policies.
Examples include NGINX, Kong, Envoy, and AWS API Gateway."

- Token scopes:

  What Are Token Scopes? (Perfect Interview Definition)

  Scopes define the specific permissions or actions that a token is allowed to perform.
  They limit the token's capabilities so even if the token is stolen, damage is contained.

  In simple words:

Scopes = "What this token is allowed to do."
They implement **least privilege** at the API level.

🟦 Where Scopes Are Used

Scopes belong to **OAuth2** and **JWT-based authorization**.
They apply to:

- Access tokens
- ID tokens (rarely)
- API keys
- Service-to-service tokens

🟩 Examples of Scopes (Interview Ready)

✓ Read-Only

read:user

read:profile

read:orders

✓ Read + Write

write:orders

write:inventory

update:profile

✓ Admin or elevated scopes

admin

superuser

manage:all

✓ Microservice-specific scopes

cdvr.read

cdvr.write

cdvr.admin

✓ Cloud & DevOps

s3:PutObject

ec2:DescribeInstances

k8s:patch:deployment

Scopes let you **slice permissions** at a fine-grained level.

🟧 Scopes in a JWT Token (Real Example)

Example JWT payload:

```
{
  "sub": "user123",
  "scope": "cdvr.read cdvr.write",
  "roles": ["adminRole#All"],
  "aud": "cdvr-api",
  "iss": "https://auth.example.com",
  "exp": 1715680000
}
```

Or the scope is represented as an array:

"scopes": ["orders.read", "orders.write"]

🟪 How Scopes Influence Access

When client hits:

GET /cdvr/admin/v1

API Gateway verifies:

- JWT signature

- Token not expired
- Scope contains required permission

  If request requires:

  cdvr.admin

  But token contains only:

  cdvr.read

  → **403 Forbidden**

  Scopes act like **API-level ACLs**.

🟥 **Token Scopes vs Roles (Important Comparison)**

Feature	**Scopes**	**Roles**
Define	Permissions or actions	User identity category
Example	orders.read	admin, user
Usage	Authorization at API route level	System access level
Issuer	Authorization Server	Identity Provider
Granularity	Fine	Coarse

Roles: who the user **is**
Scopes: what the user **can do**

Most systems use **both**.

🟦 **Scopes in OAuth2 (Important Concept)**

Clients request specific scopes:

response_type=code

scope=openid profile email cdvr.read cdvr.write

Authorization server returns only approved scopes.

Client tokens do **not** get more power than they request.

🟩 Scopes in API Gateways

Gateways enforce scopes before sending traffic to backend.

Examples:

NGINX Lua (your environment):

```
verify_scope("cdvr.admin")
```

Kong:

```
config:
  scopes_required:
    - read
    - write
```

Envoy + OPA:

Policy:

```
allow {
  input.token.scope == "orders.read"
}
```

Scopes keep enforcement **centralized at the edge**.

🟧 Why Scopes Are Critical (Security Benefits)

✓ Implement least privilege

A token with read-only scope cannot modify anything.

✓ Prevent privilege escalation

Attackers cannot use a weak token for strong operations.

✓ API surface hardening

Each endpoint requires matching scope.

✓ **Compromise isolation**

Stolen token has limited power.

✓ **Multi-tenant systems**

Tenant-specific scope patterns:

tenant:1234:read

tenant:1234:write

🟧 **Good Scope Design Practices (Interview Gold)**

- Scopes must describe **capabilities**, not resources
- Avoid extremely granular scopes (too many scopes kills UX)
- Avoid overly broad scopes (admin:all) except for ops
- Scopes must be validated at the **gateway**, not backend only
- Avoid embedding business logic into scopes
- Use consistent naming conventions:
- <service>.<resource>.<action>

  Example:

  cdvr.series.read

  cdvr.series.write

  cdvr.recording.admin

🟦 **Token Scope Enforcement Flow**

Client → API Gateway → JWT Validation → Scope Check → Backend

Backend sees only approved scopes; everything else is blocked at gateway.

🟫 **20-Second Interview Answer**

"Token scopes define what actions a token is allowed to perform.
They enforce least privilege by restricting access at the API level.
Scopes are included in the JWT/OAuth2 token claims, and the API Gateway checks whether the required scope matches the endpoint.
If the scope is missing → 403 Forbidden.
Roles identify *who* you are; scopes define *what* you are allowed to do."

- Rate limiting:

  Rate limiting restricts how many requests a client can make to an API in a given time window.
  It protects services from abuse, brute-force attacks, traffic spikes, and DoS.

- Simple meaning:
- **"You can make only X requests per Y seconds."**
- ---
- 🟦 **Why Rate Limiting Is Needed**
- Rate limiting protects against:
- Brute-force login attempts
- Token guessing attacks
- API scraping
- Bad clients overloading backend
- DoS / DDoS
- Billing explosions in cloud APIs
- Flooding microservices with too many requests
- It ensures **fair usage and system stability**.
- ---
- 🟩 **Types of Rate Limiting Algorithms**
- ✓ **1. Fixed Window**

- Easy; but allows burst at window edges.
  Example:
  100 requests per minute
- **✓ 2. Sliding Window**
- More accurate; tracks requests with timestamps.
- **✓ 3. Token Bucket (Most Popular)**
- Bucket fills with tokens → each request consumes one.
  Allows small bursts.
- **✓ 4. Leaky Bucket**
- Smooth, constant request flow (used for shaping traffic).
- ---
- 🟧 **Where Rate Limiting Happens (Important)**
- **At API Gateway**
- (NGINX, Kong, Envoy, AWS API Gateway)
  → Best place for security.
- **At the Reverse Proxy**
- → Stops abusive traffic early.
- **In Microservices**
- → Secondary fallback.
- **At CDN/Edge**
- → Stops bots globally.
- ---
- 🟪 **Rate Limiting in NGINX (Practical)**
- Define rate:
- limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
- Apply to route:
- location /api/ {

- limit_req zone=api_limit burst=20 nodelay;
- proxy_pass http://backend;
- }
- Meaning:
- 10 requests/sec allowed
- Burst of 20
- If exceeded → **429 Too Many Requests**
- ---
- 🟥 **Rate Limiting Dimensions (Very Important)**
- You can rate limit by:
- ✓ **IP address**
- Stops anonymous/attacker flooding.
- ✓ **API key**
- Per-customer throttling.
- ✓ **JWT subject (sub)**
- Per user identity.
- ✓ **Tenant ID**
- For multi-tenant SaaS.
- ✓ **Endpoint**
- Example:
- /login (strict)
- /search (loose)
- ✓ **Region**
- Example:
- India/US separated limits.

- ---
- 🟫 **Security Benefits**
- Rate limiting prevents:
- **Credential stuffing** (login brute-force)
- **API scraping / data theft**
- **Token brute-force**
- **Layer-7 DoS attacks**
- **Replay attacks**
- **Resource exhaustion**
- It ensures the API remains available under load.
- ---
- 🟦 **Real-World Example**
- **Login API — most common security pattern:**
- 5 requests / minute / IP
- 3 requests / minute / username
- Prevents bots from brute-forcing passwords.
- ---
- 🟧 **Rate Limiting vs Throttling vs Quota**

Concept	Meaning
Rate Limiting	X requests per second/minute
Throttling	Slow down requests instead of blocking
Quota	Daily/Monthly total limit

- Example quota:
- 10,000 API calls per day per API key
- ---

- 🟧 **Anti-Bypass Measures (Important)**
- Attackers try to evade limits via:
- rotating IPs
- using proxies
- sending multiple tokens
- using mobile networks
- To counter:
- combine IP + token
- stick sessions
- fingerprint clients
- use bot detection
- ---
- 🟩 **Rate Limiting in Zero Trust**
- In Zero Trust architecture:
- every request must be verified
- every identity has a limit
- rate limiting is applied per authenticated identity
- prevents insider overuse or compromised services
- ---
- 🟦 **20-Second Interview Answer**
- “Rate limiting restricts how many requests a client can make in a time window.
  It's used to prevent brute-force, DoS attacks, API abuse, and ensure fair usage.
  In gateways like NGINX we use limit_req_zone and limit_req to enforce limits per IP, token, user, or tenant, returning 429 when exceeded.
  It’s a core part of API security and availability in microservices.”
-
- Zero-trust:

What is Zero Trust? (Perfect Interview Definition)

Zero Trust is a security model that assumes no user, device, or service is inherently trusted — even if inside the network.
Every request must be authenticated, authorized, and validated continuously.

Slogan:

"Never trust. Always verify."

🟦 **Why Zero Trust Exists**

Traditional security model:
"Inside network = trusted" → totally broken now.

Attackers get in via:

- phishing
- credential theft
- VPN compromise
- lateral movement
- insider threat

So Zero Trust says:

Inside network ≠ trusted
Identity + policy define trust

🟩 **Core Principles of Zero Trust (Interview MUST-haves)**

✓ 1. Verify explicitly

Every request must authenticate:

- identity (JWT, OAuth2, mTLS)
- device posture
- workload identity (SPIFFE/SPIRE)

✓ 2. Least-privilege access

Permissions must be minimal:

- RBAC
- ABAC
- scope-based tokens

✓ 3. Assume breach

Design as if attackers are already inside:

- isolate services
- micro-segmentation
- deny-by-default

✓ 4. Continuous authentication

Not one-time login:

- short-lived tokens
- key rotation
- frequent re-auth
- session validation

✓ 5. Strong identity everywhere

Every entity has identity:

- users
- machines
- microservices
- workloads

🟧 **Key Technologies That Enable Zero Trust**

🔐 **1. Identity-Aware Proxy (API Gateway)**

Checks:

- JWT
- scopes
- roles
- device identity
- rate limits

🔐 **2. Mutual TLS (mTLS)**

Service-to-service authentication.

🔐 **3. OAuth2 / OIDC**

Strong user identity + token-based access.

🔐 **4. Short-lived tokens**

(5–15 minutes)
Limits damage if stolen.

🔐 **5. Service Mesh (Istio, Linkerd)**

Automatic:

- mTLS
- policy enforcement
- certificate rotation

🔐 **6. Device posture checks**

On mobile/desktop apps:

- OS version
- jailbroken/rooted?
- compromised?

🔐 **7. Micro-segmentation**

Restrict which service can talk to which.

🟪 **Zero Trust for Microservices (Very Important)**

Microservices environment is ZERO-TRUST by design:

✓ **Every service must authenticate via:**

- **mTLS** (identity)
- **JWT** (permissions)

✓ **Gateway enforces:**

- JWT validation
- role/scope checks
- rate limiting
- header sanitization

✓ **Mesh enforces:**

- service identity
- segmentation
- policy enforcement

Even inside Kubernetes cluster → nothing is trusted.

🟥 **Zero Trust Flow for an API Call**

Client → API Gateway → mTLS/JWT verification →

→ Authorization (role/scope) →

→ Rate limiting →

→ Backend Service

Backend also verifies identity of upstream services.

🟫 **Zero Trust vs Perimeter Security (Interview question)**

Traditional Security	**Zero Trust**
Trust internal network	Trust nothing

Traditional Security	Zero Trust
Firewall protects perimeter	Identity protects everything
Long-lived sessions	Short-lived tokens
No service-to-service auth	mTLS everywhere
Big flat network	Micro-segmentation

Zero Trust breaks the old "corporate LAN is safe" assumption.

🟧 **Zero Trust Misconceptions**

❌ **Not a product.**

Zero Trust is **a model**, not a tool.

❌ **Not only about firewalls.**

It requires IAM, API gateways, service mesh, policies.

❌ **Not only for user identity.**

Also for machine and microservice identity.

🟦 **Real-World Example**

In your cDVR system:

1. **External client** hits NGINX (gateway)
2. Gateway verifies:
    - JWT (signature + scopes)
    - roles (adminRole#All)
    - rate limiting
3. Gateway → service using mTLS
4. Service verifies gateway identity
5. Service also validates token (optional)

6. Access allowed only if all policies pass

   This is **Zero Trust** in action.

🟩 20-Second Interview Answer

"Zero Trust is a security model where no user, device, or service is trusted by default — not even inside the network.
Every request must be authenticated, authorized, and validated continuously.
It relies on strong identity (JWT, OAuth2, mTLS), least-privilege access, micro-segmentation, and continuous verification.
API gateways and service meshes implement Zero Trust in microservice environments."

- Circuit breaker (Hystrix / Resilience4j):

  What is a Circuit Breaker? (Perfect Interview Definition)

  A Circuit Breaker is a resilience pattern that stops an application from repeatedly calling a failing service, preventing cascading failures and improving system stability.

  In simple words:

  "If a service is failing, stop calling it for a while — let it recover."

🟦 Why Circuit Breakers Are Needed

Without a circuit breaker:

- Service A calls Service B
- Service B is slow / failing
- Service A keeps calling → threads pile up, timeouts accumulate
- Service A becomes slow → Service C depending on A also fails
- Entire system collapses → **cascading failure**

Circuit breakers **break the chain**.

🟩 Circuit Breaker States (Must-Mention)

Circuit breakers operate in **3 states**:

1️⃣ Closed (Normal)

- Calls flow normally
- Errors tracked behind the scenes

If **failure rate crosses threshold**, breaker trips → **Open**

2️⃣ Open (Stop Calling)

- All calls fail immediately
- No calls forwarded to the failing service
- Prevents waiting, thread blocking, resource exhaustion
- Returns fallback quickly

After **wait duration**, breaker moves to → **Half-Open**

3️⃣ Half-Open (Test Mode)

- Allows limited test requests
- If they succeed → go back to Closed
- If they fail → return to Open

🟧 Hystrix vs Resilience4j (Short comparison)

Feature	Hystrix	Resilience4j
Status	**Deprecated** (Netflix no longer maintains)	Active, modern

Feature	Hystrix	Resilience4j
Dependencies	Heavy	Lightweight, modular
Async model	Thread pools	Functional, lambdas
Integration	Spring Cloud Netflix	Spring Boot 2/3 native
Performance	Lower	Faster, no threads unless needed

Resilience4j is the successor for Java ecosystem.

🟪 **Key Features of a Circuit Breaker**

✓ **Failure Rate Threshold**

Example: failure rate > 50% over last 20 calls → Open.

✓ **Wait duration in Open State**

Example: stay Open for 60 seconds.

✓ **Permitted calls in Half-Open state**

Example: allow 5 test calls.

✓ **Timeouts**

Define how long you wait for the remote service.

✓ **Fallback mechanisms**

Return cached/empty/default response.

✓ **Bulkhead isolation**

Limit thread pools per dependency.

🟥 **Example (Resilience4j) — Interview-Level**

```
CircuitBreakerConfig config = CircuitBreakerConfig.custom()
  .failureRateThreshold(50)
```

```
  .waitDurationInOpenState(Duration.ofSeconds(30))
  .permittedNumberOfCallsInHalfOpenState(3)
  .slidingWindowSize(20)
  .build();

CircuitBreaker cb = CircuitBreaker.of("backendService", config);

Supplier<String> decorated =
  CircuitBreaker.decorateSupplier(cb, backendService::call);

Try<String> result = Try.ofSupplier(decorated)
  .recover(throwable -> "fallback-response");
```

🟦 **What Happens When Circuit Breaker Trips?**

Service A calling Service B:

1. B starts failing / timing out
2. Failure rate crosses threshold
3. Breaker goes → **Open**
4. **All requests short-circuit** (fail immediately)
5. A remains healthy
6. B recovers
7. Breaker tests → **Half-Open**
8. If OK → Closed again

This prevents **cascading failures** in distributed systems.

🟫 **Circuit Breaker with API Gateways / Service Mesh**

In modern architectures:

API Gateway:

- Can apply rate limiting + timeouts
- Circuit breakers often implemented at **downstream services** (internal)

Service Mesh (Istio/Envoy):

Istio provides circuit breakers via **DestinationRule**:

connections:

maxConnections: 100

outlierDetection:

consecutiveErrors: 5

interval: 5s

baseEjectionTime: 30s

Service Mesh moves retry/timeouts/circuit-breaker logic **out of the application code**.

🔐 **Security Relevance (Important Insight)**

Circuit breakers also help prevent security issues:

✓ **Mitigate Denial-of-Service**

Stop calling overloaded services → reduce chance of system collapse.

✓ **Prevent brute-force amplification**

A compromised service cannot flood another service.

✓ **Limit blast radius**

Zero Trust + circuit breaker isolates failure domains.

🟧 **Common Tuning Values (Be ready to discuss)**

Setting	Typical Value
failureRateThreshold	50–60%
slidingWindow	20–100 calls
waitDuration	30s–2 min
halfOpen permitted calls	1–10
timeout	1–3 seconds

🟩 **20-Second Interview Answer**

“A circuit breaker prevents cascading failures by stopping requests to a failing service.
It has three states—Closed, Open, and Half-Open.
When failures exceed a threshold, it opens and immediately fails calls instead of waiting for timeouts.
After a cool-off period it tests the service in Half-Open state.
Resilience4j is the modern Java library for this, replacing Hystrix.
Circuit breakers improve resilience, performance, and stability in distributed systems.”

- CORS security:

  What is CORS? (Perfect Interview Definition)

  CORS (Cross-Origin Resource Sharing) is a browser security mechanism that controls which external origins (domains) are allowed to access your API using frontend JavaScript.
  It prevents malicious websites from reading API responses on behalf of a logged-in user.

  In short:

  CORS = Browser saying “Which websites can talk to this API?”

  🟦 **Why CORS Exists**

Without CORS:

- Any random website could use JS to call your API
- Browser would send cookies/session automatically
- Attacker could steal user data

CORS protects users from **Cross-Site Request Forgery** and **Cross-Origin data theft**.

🟩 **Important: CORS is enforced by the browser.**

Servers don't enforce CORS → **browsers do**.

However, servers must **respond with the correct headers**.

🟧 **Key CORS Headers (Interview Must-Know)**

✓ **Access-Control-Allow-Origin**

Defines which origins are allowed.

Examples:

Access-Control-Allow-Origin: https://myapp.com

Never use:

Access-Control-Allow-Origin: *

for authenticated APIs.

✓ **Access-Control-Allow-Methods**

Allowed HTTP methods:

GET, POST, PUT, DELETE

✓ **Access-Control-Allow-Headers**

Which custom headers JS can send:

Authorization, Content-Type

✓ Access-Control-Allow-Credentials

Allows cookies / auth headers across origins:

Access-Control-Allow-Credentials: true

Only safe when used with **single allowed origin**, not *.

✓ Access-Control-Max-Age

Cache preflight requests:

86400 (1 day)

🟪 **Two Types of Requests in CORS**

1️⃣ **Simple requests**

GET/POST without special headers
→ Browser sends request normally.

2️⃣ **Preflight requests**

When request has:

- Authorization header
- JSON body
- PUT/DELETE
  → Browser sends an **OPTIONS** request first:

  OPTIONS /api

  Server must reply with CORS headers.

🟥 **CORS Threats & Security Issues**

1. Overly permissive origins

Access-Control-Allow-Origin: *

→ Means *anyone* can read sensitive API responses.

Critical vulnerability.

---

2. Allowing credentials with *

Access-Control-Allow-Origin: *

Access-Control-Allow-Credentials: true

→ **ILLEGAL** by spec and extremely dangerous.
Browser blocks it, but misconfigs leak data.

---

3. Reflection-based CORS

Server echoes Origin header:

Access-Control-Allow-Origin: <incoming origin>

If not properly verified → attacker supplies evil origin.

---

4. Failing to validate Origin header

Origin header can be spoofed **in non-browser clients**.
API should not trust it for security logic.

---

5. Confusing CORS with authentication

CORS ≠ auth.
CORS only controls browser behavior — NOTHING ELSE.

🟦 **CORS Best Practices (Gateway/Security-Level)**

✓ 1. Allow only trusted origins

Example:

Access-Control-Allow-Origin: https://app.company.com

✓ **2. Never use * for authenticated APIs**

✓ **3. Block unknown origins**

Whitelist specific domains.

✓ **4. Limit methods**

Expose only needed:

GET, POST

✓ **5. Limit headers**

Allow only necessary:

Authorization, Content-Type

✓ **6. Enable credentials only when required**

And only with strict origins.

✓ **7. Implement CORS at the API Gateway**

Examples:

- NGINX
- Kong
- Envoy

Your system uses NGINX configs:

include cors.conf;

allow all;

(“allow all” here is not CORS — it's an NGINX access directive.)

🟧 **CORS in NGINX (Relevant to You)**

A typical secure CORS config:

add_header Access-Control-Allow-Origin "https://trusted.com" always;

add_header Access-Control-Allow-Methods "GET, POST, OPTIONS" always;

```
add_header Access-Control-Allow-Headers "Authorization, Content-Type" always;
add_header Access-Control-Allow-Credentials "true" always;

if ($request_method = OPTIONS) {
   return 204;
}
```

🟫 CORS vs CSRF (Most asked in interviews)

CORS	CSRF
Browser stops JS from reading responses	Browser sends cookies automatically to other sites
Protects data theft	Protects unauthorized actions
Response protection	Request protection

CORS prevents **cross-origin data theft**
CSRF prevents **cross-origin unauthorized actions**

Both are required.

🟦 How CORS Fits in Zero Trust Architecture

Zero Trust says:

- browsers cannot be inherently trusted
- frontend origins must be validated
- token-based authentication required
- gateway sanitizes all headers

CORS enforces **origin-level identity**.

🟧 20-Second Interview Answer

"CORS is a browser security feature that controls which external domains can access your API.
It prevents malicious websites from reading sensitive API responses.
To secure CORS, we restrict allowed origins, methods, and headers, avoid * for authenticated APIs, and require strict whitelisting.
CORS is enforced by browsers, but configured at the server or API Gateway level."

Expected questions:

- *"How do you secure microservices from each other?"*
- *"What happens if auth server is down?"*

---

5. Data Security

Know:

- Encryption in transit (TLS):

  What Is Encryption in Transit? (Perfect Interview Definition)

  Encryption in transit protects data as it travels over a network by encrypting it using TLS, so that attackers cannot read, modify, or intercept the data.

  In simple words:

  TLS encrypts data when it moves between clients, servers, and microservices.

  🟦 **Why Encryption in Transit Is Required**

  Without TLS:

- Attackers can **sniff** data (MiTM)
- Passwords, tokens, and PII leak
- Traffic can be tampered with
- Replay attacks become trivial
- Session hijacking happens
- DNS spoofing leads to credential theft

Encryption in transit ensures:

- Confidentiality
- Integrity
- Authenticity

🟩 **TLS = Transport Layer Security**

TLS is the protocol behind **HTTPS**.

It ensures:

✓ **Confidentiality**

Data packets are encrypted (AES).

✓ **Integrity**

HMAC prevents tampering.

✓ **Authentication**

Server identity verified using TLS certificates.

✓ **Optional mutual authentication (mTLS)**

Both sides verify each other.

🟧 **TLS Handshake (Simplified, Interview Version)**

During TLS handshake:

1. **Client → Server**:
   ClientHello (supported ciphers, TLS version)
2. **Server → Client**:
   Sends certificate + public key
3. **Client verifies server certificate**:
   - Is it valid?
   - Is it from trusted CA?

- Is hostname correct?

4. **Client generates session key**
5. **Client encrypts session key with server's public key**
6. **Server decrypts using private key**
7. Both sides now share a **symmetric key**
8. All communication encrypted using AES (fast)

Handshake uses asymmetric crypto
Actual communication uses symmetric crypto (AES-128/256)

🟪 **TLS Versions (Security Expectation)**

- **TLS 1.3** (best, fastest, most secure)
- **TLS 1.2** (still good)
- ❌ **TLS 1.1 / 1.0** (deprecated)
- ❌ **SSLv3 / SSLv2** (completely insecure)

Interview note:

"We only allow TLS 1.2 and TLS 1.3.
Older protocols and weak cipher suites are disabled."

🟥 **Where Encryption in Transit Must Be Enforced**

✓ **Client ↔ API Gateway**

TLS (HTTPS) required.

✓ **Gateway ↔ Backend Services**

Often HTTPS or **mTLS**.

✓ **Service ↔ Service**

Usually **mTLS** through:

- NGINX

- Envoy
- Linkerd
- Istio Service Mesh

✓ **Between internal components**

- Databases
- Message queues
- Cache servers
- S3 / Cloud storage

All should use TLS.

🟫 **How NGINX Enforces Encryption in Transit**

```
server {
 listen 443 ssl;
 ssl_certificate /etc/nginx/api.crt;
 ssl_certificate_key /etc/nginx/api.key;
 ssl_protocols TLSv1.2 TLSv1.3;
}
```

Backend encryption:

```
proxy_ssl on;
proxy_ssl_certificate client.crt;
proxy_ssl_certificate_key client.key;
proxy_ssl_trusted_certificate ca.crt;
```

For **mTLS**:

```
ssl_verify_client on;
```

🟦 **Important TLS Security Configurations**

✓ **Strong cipher suites**

AES-GCM, CHACHA20
Disable:

- RC4
- 3DES
- MD5

✓ **Certificate validation**

Hostname + CA chain must match.

✓ **HSTS**

Force HTTPS:

Strict-Transport-Security: max-age=31536000

✓ **Perfect Forward Secrecy (PFS)**

Use ECDHE keys.

🟧 **Common Mistakes / Vulnerabilities**

❌ **Allowing HTTP to backend services**

→ MITM inside network

❌ **Using self-signed certs without validation**

→ Anyone can impersonate services

❌ **Weak TLS versions or cipher suites**

→ Breakable encryption

❌ **Forgetting certificate rotation**

→ Outages + compromised trust

❌ **Skipping TLS verification in API calls**

Example bug:

curl -k # insecure

verify_ssl=false # in code

🟧 **Encryption in Transit vs Encryption at Rest**

Feature	**In Transit**	**At Rest**
Protects data **while traveling**	Protects stored data	
Prevents MITM	Prevents disk theft	
Uses **TLS (HTTPS)**	Uses **AES-256, disk encryption**	
Applied to services, APIs, network	Applied to DB, file systems	

Interview trick:

Both are needed; one doesn't replace the other.

🟩 **20-Second Interview Answer**

"Encryption in transit ensures that data moving between clients and services is protected using TLS.
It provides confidentiality, integrity, and authentication so attackers cannot read or tamper with traffic.
We enforce TLS 1.2/1.3, strong cipher suites, HSTS, certificate validation, and mTLS for service-to-service calls.
Combined with good certificate management, it forms the foundation of secure microservice communication."

- Encryption at rest (DB-level):

  What Is Encryption at Rest? (Perfect Interview Definition)

Encryption at rest protects stored data (database files, backups, disks, logs) by encrypting it using strong cryptographic keys, so that if the underlying storage or disks are accessed or stolen, data remains unreadable.

In simple words:

“Data stored on disk is encrypted, so even if someone steals the DB file, they cannot read it.”

🟦 Why Encryption at Rest Is Needed

Without encryption at rest:

- Stolen database files = data breach
- Administrators with disk-level access can read sensitive data
- Cloud providers can accidentally expose volumes
- Attackers who obtain backups can extract PII
- Compliance violations (GDPR, HIPAA, PCI)

Encryption at rest protects against:

- Disk theft
- Snapshot exposure
- Backup compromise
- Insider threats
- Physical access attacks

🟩 Two Levels of Encryption at Rest

1️⃣ Disk-level Encryption (Full-Disk Encryption: FDE)

Encrypts:

- files
- logs
- DB data files

- indexes
- temporary files
- swap memory

Using:

- LUKS
- dm-crypt
- BitLocker
- AWS EBS encryption
- GCP persistent disk encryption

Pros: Easy, transparent to DB
Cons: DBA can still read data if OS compromised

2️⃣ Database-Level Encryption

Database encrypts data **inside the database engine**, not only on disk.

Includes:

- **TDE (Transparent Data Encryption)**
- **Column-level encryption**
- **Field-level encryption (application-managed)**

Let's break these.

🟧 Transparent Data Encryption (TDE)

TDE encrypts data files automatically, using a **master key inside the DB engine**.

Supported by:

- MySQL Enterprise
- PostgreSQL pgcrypto
- Oracle TDE

- SQL Server TDE
- MongoDB Enterprise encryption at rest
- AWS RDS encryption

  What it encrypts:

- data files
- backup files
- WAL/redo logs
- temp files

  What it **doesn't** protect:

- Data in memory
- Data transmitted to client
- DBAs with SELECT access
- App-level misuse

  TDE protects against **disk theft**, not **privilege misuse**.

🟪 Column-Level Encryption

Encrypt only sensitive fields:

- password
- credit card
- SSN
- personal info

  DB driver decrypts fields on SELECT.

  Example in PostgreSQL pgcrypto:

  SELECT pgp_sym_encrypt('4111111111111111', 'key');

  Pros:

- Fine-grained

- Protects DBAs without access

Cons:

- Queries cannot search encrypted columns unless using deterministic encryption

---

🟥 **Application-Level Encryption (Most Secure)**

Application encrypts data **before sending to DB**:

- Service generates encryption key
- Encrypts sensitive fields
- Stores ciphertext in DB
- DB never sees plaintext

Pros:

- Maximum security
- Protects against malicious DBAs
- Protects DB compromise

Cons:

- Harder to index/search
- Requires key management

Used for:

- Credit card PCI
- Financial records
- Medical records (HIPAA)

🟫 **Key Management (Critical Interview Topic)**

Encryption is **useless without secure key management**.

Keys must be:

- Stored in **KMS** (AWS KMS, GCP KMS, Azure Vault)

- Rotated periodically
- Not embedded in code
- Separated from data
- Enforced with IAM permissions

Hierarchy:

1. **Master Key (CMK)** – in KMS
2. **Data Encryption Key (DEK)** – used to encrypt actual data
3. DB encrypts DEKs with CMK
4. DEKs decrypt data files

This is called **Envelope Encryption**.

🟦 **Encryption at Rest vs Encryption in Transit**

Aspect	**Encryption in Transit**	**Encryption at Rest**
Protects	Data *moving* over network	Data *stored* on disk
Uses	TLS	AES-256
Stops	MITM, snooping	Disk theft, backup exposure
Keys	TLS certificates	KMS-managed DEKs
Applies to	API Gateway, services	DB, EBS/SSD, snapshots

Both required in modern security.

🟧 **Common Mistakes / Vulnerabilities**

❌ **Storing keys in:**

- code
- GitHub
- config files

- environment variables

❌ **Not encrypting backups**

Backups are the #1 source of breaches.

❌ **Relying only on disk encryption**

Does not protect against:

- SQL injection
- DB admin misuse
- compromised service

❌ **No rotation**

Keys should be rotated at least yearly (or automatically via KMS).

🟧 **How Cloud Providers Implement This (Expected Knowledge)**

AWS:

- EBS encryption
- RDS encryption
- S3 SSE-KMS
- DynamoDB encryption at rest
- KMS key hierarchy

GCP:

- CMEK / CSEK
- Cloud SQL encryption
- GCS encryption

Azure:

- Storage Service Encryption
- SQL TDE
- Managed HSM

You only need to say this in interviews:

“Cloud platforms encrypt data using AES-256 with envelope encryption and manage keys using KMS.”

🟩 **20-Second Interview Answer**

“Encryption at rest protects stored data in databases, volumes, and backups using strong encryption like AES-256.
It prevents attackers or insiders from reading data even if they obtain the raw DB files or disk snapshots.
It includes disk encryption, TDE, column-level encryption, and application-level encryption.
Key management is critical — keys must be stored in KMS, rotated, and never embedded in code.”

- Password hashing (bcrypt/argon2):

  What Is Password Hashing? (Perfect Interview Definition)

  Password hashing is the process of converting a user password into a one-way, irreversible string using a slow, salted hash function designed to resist brute-force attacks.

  Key idea:

  Passwords must NEVER be stored in plaintext or with fast hashes (SHA256/MD5).

🟦 **Why Normal Hashes (SHA256/MD5) Are NOT Safe**

Because:

- They are **too fast**
- GPU can brute-force billions of hashes/sec
- Rainbow tables can crack unsalted hashes
- Attackers can reverse them easily

We need a **slow, memory-hard hashing algorithm** → bcrypt / Argon2.

🟩 Characteristics of Secure Password Hashing Algorithms

1. **Salted**
   Prevents rainbow table attacks.
   Each user's password uses a unique random salt.
2. **Slow**
   Expensive to compute (intentional).
   Slows down brute-force attempts.
3. **Memory-hard** (Argon2)
   Requires large memory → makes GPU attacks difficult.
4. **Configurable work factor**
   Allows increasing cost as hardware improves.

🟧 bcrypt — Most Common Password Hashing Algorithm

✓ Features:

- Salt included internally
- Configurable cost factor (2^cost)
- Resistant to brute-force
- Slow by design
- 60-character output

Example hash:

$2b$12$Ck8JY6ZYq9R0D7jS9rVh8Olxf3dJyXiBQkAuj...

Breakdown:

- $2b$ → bcrypt version
- 12 → cost factor
- rest → salt + hashed password

How bcrypt works:

- Generates a **random salt** (16 bytes)
- Applies **Blowfish-based KDF**
- Outputs combined salt + hash

🟪 **Argon2 — Modern, Strongest Choice (Recommended)**

Winner of the **Password Hashing Competition (PHC)**.

Variants:

- **Argon2id** (BEST — hybrid of i + d)
- Argon2i (CPU-hard)
- Argon2d (GPU-hard)

✓ Strengths:

- Memory-hard
- Highly configurable
- Defends against GPU cracking
- Tunable: memory, iterations, parallelism

Example parameters:

memory: 256MB

iterations: 3

parallelism: 4 threads

Argon2id is the standard for new systems.

🟥 **Password Hashing Flow (This is interview gold)**

1. User creates password

"Hello@123"

2. Backend generates random salt

e.g., 16–32 bytes.

3. Hash = bcrypt/argon2(password + salt)

4. Store hash + salt in DB

Not the password.

5. During login

- Backend retrieves stored salt+hash
- Hashes user input
- Compares constant-time (avoid timing attacks)

If match → login success.

🟫 Why Salting Matters

Without salt:

MD5("password") → always same output

Attackers precompute billions of hashes → instant crack.

With salt:

bcrypt("password" + 16-byte unique salt) → unique per user

Even if two users have same password → hashes differ.

🟦 Cost Factors / Work Parameters

bcrypt:

- cost = 10–14
  Higher = slower = safer

Argon2:

- t = iterations
- m = memory cost
- p = parallelism

Recommended:

Argon2id: t=2, m=256MB, p=2

🟨 **Why Use Slow Hashes?**

If you use fast hashes (e.g., SHA256):

Attacker with GPU:

10 billion guesses per second

bcrypt/argon2:

~100 guesses per second

Slower = safer
That's the whole point.

🟧 **Password Hashing vs Encryption (Critical Distinction)**

Hashing	**Encryption**
One-way	Two-way
Cannot decrypt	Can decrypt
Used for passwords	Used for data protection
Uses salt	Uses keys

Passwords must **never** be encrypted — always hashed.

🟩 **Password Hashing Vulnerabilities to Avoid**

❌ **MD5/SHA1/SHA256 for passwords**

(Too fast → cracked instantly)

❌ **Same salt for all users**

(Rainbow tables become possible)

❌ **No salt**

(Worst possible design)

❌ **Storing plaintext passwords**

(Critical vulnerability)

❌ **Storing encryption keys near hashes**

(Pointless encryption)

❌ **Comparing hashes using normal equality**

(Leads to timing attacks)

🟥 **Short Example (bcrypt in Java)**

String hash = BCrypt.hashpw(password, BCrypt.gensalt(12));

boolean matches = BCrypt.checkpw(password, hash);

🟪 **Short Example (Argon2 in Java)**

Argon2 argon2 = Argon2Factory.create();

String hash = argon2.hash(3, 65536, 1, password);

boolean matches = argon2.verify(hash, password);

🟦 **20-Second Interview Answer**

“Password hashing means storing passwords using one-way, slow, salted hashing algorithms like bcrypt or Argon2.
These algorithms deliberately slow down brute-force attacks and include salts to prevent rainbow table attacks.
Argon2id is the strongest modern choice because it is memory-hard and GPU-resistant.

On login, we rehash the input and compare it to the stored hash using constant-time comparison.”

- Secure secret storage:

  What Is Secure Secret Storage? (Perfect Interview Definition)

  Secure secret storage refers to safely storing sensitive application secrets (API keys, DB passwords, certificates, JWT signing keys, OAuth client secrets, encryption keys) using dedicated secure systems such as Vault, KMS, or Secrets Managers — never in code, repos, or config files.

- In simple terms:
- **Secrets must be stored encrypted, access-controlled, and rotated — NOT in plain text.**
- ---
- 🟦 **What Counts as a Secret?**
- Database passwords
- API keys
- JWT signing keys (HS256)
- Private keys (RSA, EC)
- OAuth2 client secrets
- Service-to-service tokens
- AWS/GCP/API credentials
- Encryption keys (DEK/CMK)
- SSH keys
- Third-party access tokens
- Basically **anything that grants access**.
- ---
- 🟩 **Goals of Secure Secret Storage**

- ✓ **Confidentiality**
- Secrets are encrypted on disk.
- ✓ **Integrity**
- Tampering is detectable.
- ✓ **Access control**
- Only authorized apps can read secret.
- ✓ **Versioning**
- Track changes.
- ✓ **Rotation**
- Regularly rotate keys.
- ✓ **Audit Logging**
- Every access is logged.
- ✓ **Eliminate hardcoding**
- No secrets in:
- Git
- Docker images
- NGINX files
- ENV vars (with caution)
- Kubernetes manifests
- Config files stored on disk
- ---
- 🟧 **Where Secrets Should NEVER Be Stored**
- ❌ **Hardcoded in source code**
- (Leaked if repo compromised)
- ❌ **Stored in GitHub/GitLab**

- Even private repos.
- ❌ **In docker images**
- Image layers become permanent.
- ❌ **In plain-text config files**
- Easily leaked through CI/CD.
- ❌ **In frontend code**
- Browser can see it.
- ❌ **Public S3 buckets**
- Seen in many breaches.
- ---
- 🟪 **Industry-Standard Secure Secret Storage Tools**
- **1. HashiCorp Vault**
- Most powerful, enterprise-grade secrets manager.
- Features:
- Dynamic DB credentials
- PKI engine (generate mTLS certs)
- Lease & renewal
- Policies
- Audit log
- Transit encryption engine
- ---
- **2. Cloud Provider Secret Managers**
- **AWS Secrets Manager**
- **AWS SSM Parameter Store**
- **GCP Secret Manager**

- **Azure Key Vault**
- Features:
- Encryption by KMS
- IAM-based access
- Versioning
- Rotation
- ---
- **3. KMS (Key Management Service)**
- AWS KMS / GCP KMS / Azure Key Vault.
- Used for:
- Master encryption keys (CMK)
- Envelope encryption
- DB encryption keys
- Signing keys
- ---
- **4. Kubernetes Secrets (BUT carefully)**
- By default → Base64 encoded (NOT encrypted).
  Must enable:
- Encryption at rest
- RBAC
- sealed-secrets or external secret operator
- ---
- 🟥 **How Secrets Are Protected (Mechanisms)**
- ✓ **Encryption-at-rest**
- AES-256 using CMK stored in KMS/HSM.
- ✓ **Encryption-in-transit**

- TLS/mTLS when retrieving secrets.
- ✓ **Access control**
- Fine-grained IAM policies:
- which app
- which service account
- which environment
- read-only or write
- ✓ **Rotation & versioning**
- Auto-rotate DB passwords / API keys.
- ✓ **Dynamic secrets (Vault)**
- Secret generated on demand; expires automatically.
- ---
- 🟫 **How Applications Fetch Secrets Securely**
- **1. At runtime using IAM identity**
- App retrieves secrets from:
- Vault
- AWS Secrets Manager
  Devices authenticate via:
- IAM role
- Kubernetes Service Account
- mTLS client cert
- No static credentials needed.
- ---
- **2. Sidecar Injection (Vault Agent / Kubernetes CSI driver)**
- Secrets injected into:
- memory

- tmpfs file
  NEVER stored on disk.
- ---
- **3. Environment variables (with caution)**
- Acceptable for:
- short-lived containers
- non-sensitive secrets
  But:
- logs may leak
- process list may leak
- not good for long-term storage
- ---
- 🟦 **Secure Secret Storage Architecture (Interview-ready)**
- App → Auth (IAM / JWT / mTLS)
- ↓
- Secrets Manager / Vault
- ↓ (AES-256 encrypted storage)
- Return secret → App
- ↓
- App uses secret in memory
- Secrets never:
- hit Git
- appear in logs
- are stored on disk
- ---
- 🟧 **Common Mistakes (Important)**

- ❌ **Hardcoding secrets in code**
- ❌ **Committing secrets to Git**
- ❌ **Using .env files in production**
- ❌ **Logging secrets**
- ❌ **Using long-lived credentials**
- ❌ **Using same secret in multiple environments**
- ❌ **Storing secrets in CI/CD variables without protection**
- ❌ **Not rotating keys**
- ---
- 🟧 **Best Practices for Secure Secret Storage**
- ✓ **Short-lived tokens (temporary credentials)**
- ✓ **Rotate secrets automatically**
- ✓ **Use IAM-controlled access**
- ✓ **Encrypt secrets at rest (AES-256)**
- ✓ **Enforce TLS/mTLS**
- ✓ **Do not store secrets in environment variables long term**
- ✓ **Do not store secrets on disk**
- ✓ **Use HSM/KMS for master keys**
- ✓ **Use Vault for dynamic secrets**
- ✓ **Least privilege access**
- ---
- 🟦 **20-Second Interview Answer**
- "Secure secret storage means storing access credentials, keys, and API secrets in a secure, encrypted secrets manager instead of code or config files.
  We use Vault or cloud secrets managers, enforce encryption-at-rest with KMS, apply

IAM-based access control, rotate secrets regularly, and ensure apps retrieve secrets securely at runtime.
Secrets must never be hardcoded or stored in Git."

-

Expected questions:

- *"AES vs RSA — when do you use which?"*
- *"How do you store passwords?"*

---

6. CI/CD Security

Know:

- Secret masking
- Dependency scanning
- Docker image scanning
- Artifact signing
- No credentials in repo

Expected questions:

- *"How do you secure CI/CD pipelines?"*

🔥 **PART 2 — Essential 30-sec answer templates**

Here are ready-to-speak answers.

⭐ **1. JWT vs OAuth2 vs OIDC**

OAuth2 handles authorization and issues access tokens.

OIDC sits on top of OAuth2 and adds authentication using ID Tokens and discovery.

JWT is just the format used for tokens – RS256/HS256 signatures allow stateless validation.

⭐ 2. RS256 vs HS256

HS256 uses one shared secret to sign and verify. Anyone with the secret can forge tokens.

RS256 uses private key to sign and public key to verify. Only the private key holder can sign.

⭐ 3. How do you secure NGINX endpoints?

Use TLS, rate limiting, header filtering, disable weak ciphers, validate JWT in access_by_lua,

apply CORS rules, hide server tokens, use strict timeouts, and restrict upstream exposure.

⭐ 4. SQL Injection prevention

Never build queries with string concatenation. Use prepared statements, parameterized queries,

and least-privilege DB users. Validate input and avoid dynamic query parts.

⭐ 5. XSS prevention

Encode all output, sanitize user inputs, use Content-Security-Policy headers,

and never trust HTML passed from the client.

⭐ 6. CSRF prevention

Use same-site cookies, CSRF tokens, and double-submit cookie technique.

For APIs, prefer stateless JWT + CORS rules.

⭐ 7. Microservice-to-microservice security

Use mTLS for identity, JWT or opaque tokens for authorization, rate limiting,

and follow zero-trust – never trust traffic inside the cluster.

⭐ 8. How do you revoke JWTs?

Use short expiration and refresh tokens, or maintain a server-side blacklist keyed by jti.

Opaque tokens solve revocation by using introspection.

🔥 PART 3 — One-page summary to memorize before Monday

Authentication

- OAuth2 = authorization
- OIDC = authentication
- JWT = token format
- RS256 = public/private
- HS256 = shared secret

Authorization

- RBAC, scopes, claims in JWT
- Role checks at gateway or microservice

OWASP

- SQLi
- XSS
- CSRF
- Sensitive Data Exposure
- Broken Access Control
- SSRF

NGINX

- access_by_lua_block
- auth_request
- upstreams
- rate limiting

- proxy headers
- CORS

Crypto

- RSA = asymmetric
- AES = symmetric
- Hashing = bcrypt

Microservices

- mTLS
- API Gateway
- Request signing

CI/CD

- Secrets mask
- Dependency scanning
- Signed artifacts
- No secrets in repo

🔥 **PART 4 — Final mock interview (5 min version)**

1. What is OIDC?

OpenID Connect is an identity layer on top of OAuth2 that provides authentication using ID Tokens, discovery, and JWKS for signature verification.

2. What happens when a JWT hits your NGINX?

Access phase Lua reads Authorization header → extracts token → loads JWKS → verifies RS256 signature → checks roles → allow/deny.

3. How do you secure a microservice system?

mTLS, JWT validation, gateway-based auth, rate limiting, secret management, DB least privilege, secure CI/CD, detailed audit logging.

4. What's the difference between SQL injection and NoSQL injection?

Both exploit unsafe query building. SQL injection targets SQL syntax; NoSQL injection targets JSON-like queries (Mongo, Neo4j). Prevention: parameterized queries.

5. How do you protect APIs from CSRF?

SameSite cookies, CSRF tokens, or using Authorization header-based JWTs instead of cookie-based sessions.

🔥 **PART 5 — 2-day study plan (Saturday + Sunday)**

Saturday

- JWT, OAuth2, OIDC
- NGINX security
- Hands-on: decode 5 JWT tokens, validate public key
- Review OWASP Top 10

Sunday

- Microservices security
- CI/CD security
- Secure coding in Java
- Mock interview round
- Quick revision of NGINX configs